Building AI Agents Architecture

https://gemini.google.com/app/a9ccaf12937eb006

User prompt: Can we make AI Agent

Response: Yes, you can build an AI agent—ranging from a simple rule-guided LLM wrapper to an autonomous multi-agent system capable of using external tools and executing complex workflows.

Core Architecture of an AI Agent

Every modern AI agent typically consists of four foundational layers:

1. **The Brain (LLM / Reasoning Engine):** Models like Claude 3.5 Sonnet, GPT-4o, or open-weight models (Llama 3.3, DeepSeek) interpret prompts, generate action plans, and evaluate results using reasoning patterns like **ReAct** (Reasoning + Acting) or **Plan-and-Solve**.
2. **Memory:**
    - *Short-term:* In-context conversation history, current task state, and intermediate reasoning steps.
    - *Long-term:* Vector databases (e.g., Pinecone, Qdrant, Chroma, pgvector) to store and retrieve historical context and documents via semantic search (RAG).
3. **Tools & Function Calling:** APIs, bash shells, web search engines, Python REPLs, and database connectors that allow the agent to read or write data outside its static weights.
4. **Planning & Control Loop:** The execution loop that evaluates the output of each tool call against the user's objective, decides whether the task is complete, or recurses to correct errors.

Popular Frameworks to Build One

Depending on your programming stack and use case:

- **Python:**
    - **LangGraph (by LangChain):** Best for cyclic, stateful, and highly controlled graph-based agent workflows.
    - **CrewAI / AutoGen:** Designed for multi-agent collaboration where distinct agents (e.g., researcher, writer, reviewer) coordinate on tasks.
    - **LlamaIndex:** Best when your agent heavily relies on data retrieval and document querying.
- **TypeScript / Node.js:**
    - **Vercel AI SDK:** Clean primitives for streaming tool calls and multi-step agent loops.
    - **LangGraph.js:** The JS/TS port of LangGraph for graph-based orchestrations.

Basic Build Blueprint (Python)

A minimal, functional tool-calling agent using Python and an API client:

```
import os
from openai import OpenAI

client = OpenAI(api_key=os.environ.get("OPENAI_API_KEY"))

# 1. Define tools the agent can execute
tools = [
    {
        "type": "function",
        "function": {
            "name": "run_system_command",
            "description": "Executes a shell command to gather system diagnostic data.",
            "parameters": {
                "type": "object",
                "properties": {
                    "command": {"type": "string", "description": "The shell command to run"}
                },
                "required": ["command"]
            }
        }
    }
]

# 2. Main agent control loop
messages = [{"role": "user", "content": "Check disk space on this machine."}]

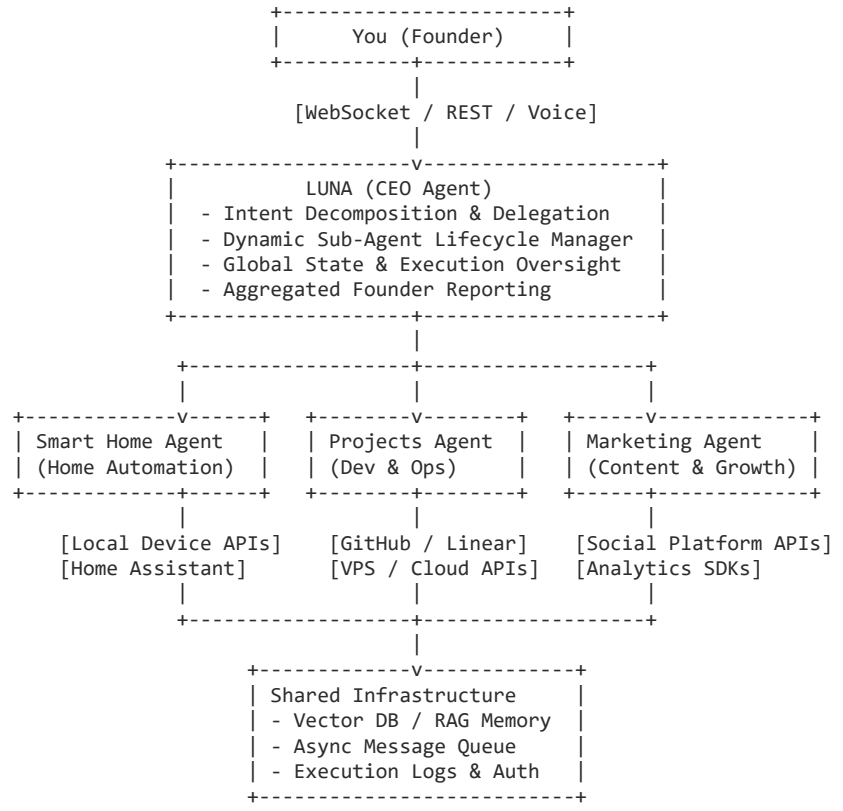
response = client.chat.completions.create(
    model="gpt-4o",
    messages=messages,
    tools=tools
)
```

```
# 3. Handle tool calls and return results back to the model
message = response.choices[0].message
if message.tool_calls:
    print(f"Agent requested tool: {message.tool_calls[0].function.name}")
    # Execute tool logic here and append the output back to `messages` for final evaluation
```

User prompt: Make me Architecture for Main Agent as a CEO who will report me everthing and i will ask him what ever i want base on my task other agent will work supervice by CEO basicali i want My Smart home app to be contraol, and my projact to be hadel my social media marketing everthink i want i will ask CEO then he will create support agent as he want they will work for me will call this application name as a LUNA

Response: Here is an end-to-end, production-ready system architecture for **LUNA**—an autonomous executive agent operating as your CEO to supervise dynamic, domain-specific sub-agents across home automation, project management, and social media marketing.

High-Level Architecture Diagram



Core Structural Layers

1. Interface & Communication Layer

- **Founder Touchpoints:** A unified mobile/desktop dashboard (Next.js/React) backed by WebSockets for real-time streaming, status feeds, and push notifications.
- **CEO Channel:** You interact only with LUNA. You issue commands ("*Prep a campaign for my new tool launch, check project build statuses, and dim the office lights to 40%*"), and LUNA streams back conversational updates and consolidated debriefs.

2. The Orchestrator (LUNA CEO Agent)

- **Goal & Intent Planner:** Uses high-parameter frontier reasoning (Claude 3.5 Sonnet / GPT-4o) with structured outputs (JSON schema) to deconstruct broad requests into discrete multi-step dependency graphs.
- **Dynamic Agent Spawner:** When an objective requires a specialist, LUNA either routes the work to persistent workers or instantiates transient sub-agents on the fly with specialized prompts, scoped permissions, and dedicated tools.
- **Validation & Synthesis Loop:** LUNA reviews outputs from sub-agents. If an agent fails a task, LUNA intervenes, adjusts context, or retries before compiling the final operational briefing for you.

3. Domain Worker Agents

Agent Domain	Primary Capabilities	Integrations / Toolsets
Smart Home Manager	Reads IoT telemetry, controls lighting, locks, thermostats, and triggers contextual scenes.	Home Assistant API, MQTT, Matter/Zigbee bridges, Local LAN webhooks.

Agent Domain	Primary Capabilities	Integrations / Toolsets
Project & Tech Lead	Tracks project roadmaps, checks Git repos, spins up test containers, triggers deployments, reviews build logs.	GitHub/GitLab REST API, Linear/Jira APIs, Docker/VPS SSH executors, Cloudflare/Vercel APIs.
Social Media & Growth	Drafts posts, formats assets, tracks engagement metrics, schedules releases across platforms.	X/Twitter API, YouTube Data API, Meta Graph API, Buffer/PostHog analytics.
Ad-Hoc Spawners	Temporary workers created on-demand for research, data scraping, code debugging, or asset generation.	Headless browsers (Playwright), Python REPL sandboxes, Web Search APIs.

4. Shared Infrastructure & Persistence

- **State & Message Broker:** Redis Pub/Sub or Celery/Temporal for async execution queues, ensuring sub-agent operations do not block LUNA's main conversation loop.
- **Unified Memory System:**
  - *Operational Memory (PostgreSQL / SQLite):* Active tasks, sub-agent IDs, audit logs, and schedule states.
  - *Semantic Memory (Qdrant / pgvector):* Founder preferences, historical decisions, API schemas, and context embeddings.
- **Security & Permission Guardrails:** A role-based access control (RBAC) gateway where destructive actions (e.g., executing raw server migrations, deploying untested code, unlocking external security doors) pause for CEO-level verification and founder sign-off.

Technical Implementation Blueprint

The fastest and most stable way to implement LUNA is using **LangGraph** (Python or TypeScript) due to its first-class support for hierarchical multi-agent state machines, cyclical control loops, and dynamic subgraphs.

State Machine & Delegation Schema

```
from typing import TypedDict, List, Dict, Any, Optional
from pydantic import BaseModel, Field

# 1. Structured Output from LUNA's Planning Engine
class SubTask(BaseModel):
    task_id: str
    target_domain: str = Field(description="smart_home, projects, marketing, or general_adhoc")
    instruction: str
    dependencies: List[str] = Field(default_factory=list)

class CEODecision(BaseModel):
    acknowledgement_to_founder: str
    subtasks: List[SubTask]
    requires_confirmation: bool = False
    confirmation_reason: Optional[str] = None

# 2. Global Shared Graph State
class LunaGlobalState(TypedDict):
    founder_input: str
    active_plan: Optional[CEODecision]
    domain_results: Dict[str, Any]
    ceo_final_report: str
```

CEO Agent Control Logic (Python / LangGraph Concept)

```
import json
from openai import OpenAI

client = OpenAI()

CEO_SYSTEM_PROMPT = """
You are LUNA, the Chief Executive AI Agent.
You answer directly to the Founder. You oversee specialized sub-agents:
1. smart_home: Handles IoT, lighting, environment, security, automation.
2. projects: Handles coding tasks, servers, builds, roadmaps, tickets.
3. marketing: Handles content creation, social media scheduling, platform analytics.

When the Founder gives a command:
- Decompose the goal into targeted sub-agent tasks.
- If an action involves critical risk (e.g., prod deployments, critical security changes), flag requires_confirmatio
- Provide a concise executive acknowledgement.
"""

def luna_plan_and_delegate(state: LunaGlobalState) -> LunaGlobalState:
    founder_message = state["founder_input"]

    # Generate execution schema
```

```
    completion = client.beta.chat.completions.parse(
        model="gpt-4o",
        messages=[
            {"role": "system", "content": CEO_SYSTEM_PROMPT},
            {"role": "user", "content": founder_message}
        ],
        response_format=CEODecision
    )

    plan: CEODecision = completion.choices[0].message.parsed
    state["active_plan"] = plan
    return state

def execute_subtasks(state: LunaGlobalState) -> LunaGlobalState:
    plan = state["active_plan"]
    results = {}

    for subtask in plan.subtasks:
        if subtask.target_domain == "smart_home":
            # Invoke Smart Home Worker (e.g., Home Assistant REST call)
            results[subtask.task_id] = f"Home automation adjusted: {subtask.instruction}"
        elif subtask.target_domain == "projects":
            # Invoke Project Worker (e.g., check Git repos, deploy status)
            results[subtask.task_id] = f"Project ops executed: {subtask.instruction}"
        elif subtask.target_domain == "marketing":
            # Invoke Marketing Worker (e.g., generate post draft, query metrics)
            results[subtask.task_id] = f"Marketing pipeline updated: {subtask.instruction}"
        else:
            results[subtask.task_id] = f"Generic worker executed: {subtask.instruction}"

    state["domain_results"] = results
    return state

def luna_synthesize_report(state: LunaGlobalState) -> LunaGlobalState:
    founder_message = state["founder_input"]
    results = state["domain_results"]

    # LUNA summarizes outputs into an executive brief
    synthesis_prompt = f"""
    Founder Request: {founder_message}
    Execution Results: {json.dumps(results)}

    Generate an executive CEO briefing back to the founder. Keep it crisp, factual, and actionable.
    """

    summary = client.chat.completions.create(
        model="gpt-4o",
        messages=[
            {"role": "system", "content": "You are LUNA reporting back completed operations to the Founder."},
            {"role": "user", "content": synthesis_prompt}
        ]
    )

    state["ceo_final_report"] = summary.choices[0].message.content
    return state
```

Suggested Technology Stack

- **Core Framework:** Python 3.11+ with **LangGraph** (cyclic supervisor-worker model) or **FastAPI** serving a headless event loop.
- **Agent Brains:** Claude 3.5 Sonnet (for CEO orchestration, code, complex problem solving) + GPT-4o-mini (cost-effective worker agent execution).
- **Data & Tool Connectors:**
  - **Smart Home:** Official Home Assistant Webhook / REST API (`requests` / `aiohttp`).
  - **Projects:** PyGithub, Docker SDK for Python, Linear GraphQL API.
  - **Marketing:** Meta Graph API, Tweepy (X API), Playwright (for automated asset validation and scraping).
- **Database & Memory:** PostgreSQL (via Supabase or local Docker) for structured state, and Qdrant / ChromaDB for long-term semantic context.
